

Alexandre Tolomeotti Enokida
João Pedro da Silva de Andrade

***Benchmark* e Análise Empírica de Algoritmos de Ordenação**

Relatório técnico de atividade prática solicitado pelo professor André Luiz Satoshi Kawamoto na disciplina de Estrutura de Dados do Programa de Pós-Graduação em Ciência da Computação da Universidade Tecnológica Federal do Paraná.

Universidade Tecnológica Federal do Paraná – UTFPR

Departamento Acadêmico de Computação – DACOM

Programa de Pós-Graduação em Ciência da Computação – PPGCC-CM

Campo Mourão

Julho / 2026

Resumo

Neste trabalho, comparamos empiricamente o desempenho de seis algoritmos clássicos de ordenação: Bubblesort, Insertionsort, Selectionsort, Heapsort, Quicksort e Shellsort. A partir da coleta de dados de execução (e.g., quantidade de comparações e tempo de execução), analisamos os comportamentos dos algoritmos de ordenação em diferentes cenários de entrada.

Sumário

1	Materiais e Métodos	4
2	Experimento	4
3	Declaração do Uso de Inteligência Artificial Generativa	5

1 Materiais e Métodos

Os experimentos deste trabalho foram executados em um *desktop* (WSL) equipado com um processador Intel Core i5-10400F (6 núcleos, 12 *threads*, 2.9 GHz, 12 MB cache), 16 GB de memória RAM DDR4-2666, GPU NVIDIA GeForce RTX 3060 (12 GB) e um dispositivo de armazenamento NVMe SSD de 1 TB.

Realizamos três experimentos, todos de acordo com as especificações propostas:

1. Execução dos algoritmos de custo quadrático (Bubblesort, Selectionsort e Insertionsort) com um vetor de entrada de 30.000 elementos, variando instâncias em que a entrada é aleatória, ordenada de forma crescente e ordenada de forma decrescente.
2. Execução do Heapsort e Quicksort (utilizando o último elemento como pivô) com um vetor de entrada de 5.000 elementos ordenado de forma crescente. Além disso, neste experimento realizamos 10 repetições.
3. Execução do Insertionsort e Shellsort com um vetor de entrada de 50.000 elementos parcialmente ordenado (aprox. 5% do total ordenado).

2 Experimento

A Tabela 1 apresenta os resultados do primeiro experimento. No vetor aleatório, o Selectionsort foi mais rápido que o Insertionsort — suspeitamos que seja por causa da instância, i.e., o vetor aleatório gerado, o que causa diferença devido a suas lógicas de comparação. O Selectionsort é invariante, executando sempre todas as 449.985.000 comparações para encontrar o menor elemento de cada iteração. Já o Insertionsort possui comportamento adaptativo e interrompe as comparações internas assim que encontra a posição correta do elemento avaliado, o que reduz suas comparações pela metade no cenário aleatório.

Tabela 1 – Resultados da execução do primeiro experimento.

Vetor de entrada	Algoritmo	Comparações	Trocas	Tempo (s)
Aleatório	Bubblesort	449.779.441	226.281.788	1,07
	Insertionsort	226.311.773	226.281.788	0,31
	Selectionsort	449.985.000	29.991	0,22
Ordenado (crescente)	Bubblesort	29.999	0	0,00002
	Insertionsort	29.999	0	0,00004
	Selectionsort	449.985.000	0	0,23170
Ordenado (decrescente)	Bubblesort	449.985.000	449.985.000	0,55845
	Insertionsort	449.985.000	449.985.000	0,59881
	Selectionsort	449.985.000	15.000	0,22047

A Figura 1 apresenta os resultados do segundo experimento. O Quicksort clássico com pivô no último elemento possui comportamento quadrático $O(n^2)$ em vetores ordenados, pois gera partições desbalanceadas de tamanhos $N - 1$ e 0 , resultando em aproximadamente 12,5 milhões de comparações. Esse comportamento anula a divisão eficiente do problema e maximiza as chamadas recursivas na pilha. Contrariamente, o Heapsort sempre mantém uma estrutura de árvore binária, obtendo comportamento linearítmico $O(n \log n)$ em qualquer cenário, e refletindo em um tempo médio significativamente inferior.

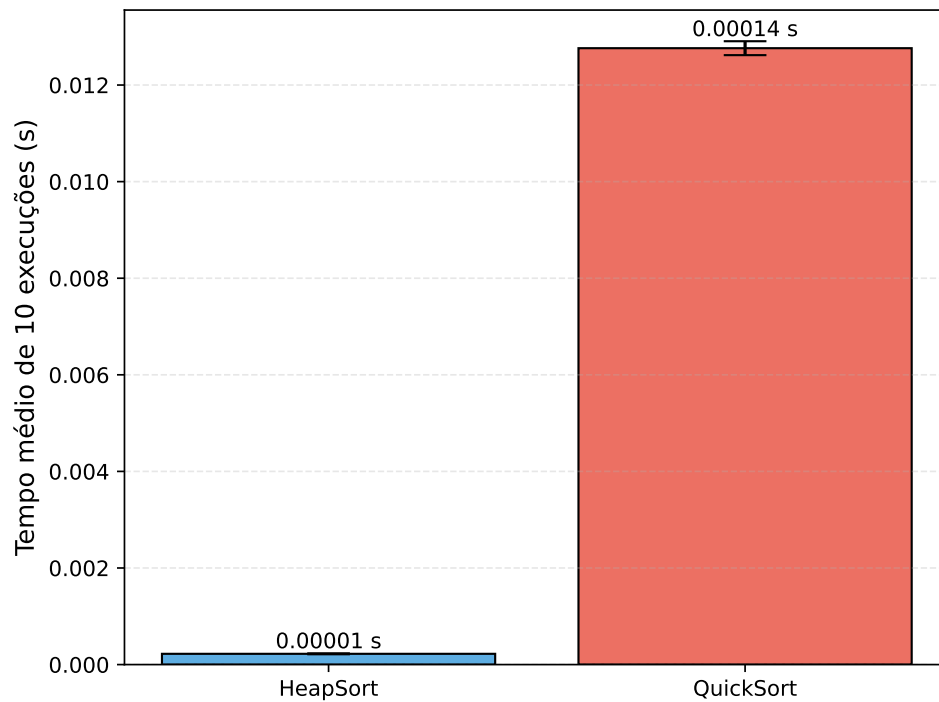


Figura 1 – Tempo médio de 10 execuções do Heapsort e Quicksort com um vetor de 5.000 elementos ordenado de forma crescente.

Por fim, a Tabela 2 apresenta os resultados do terceiro experimento. Neste cenário, o Insertionsort apresentou melhor desempenho que o Shellsort, conforme demonstram os tempos de execução. Como o Insertionsort possui complexidade linear $O(n)$ em vetores quase ordenados, ele realiza poucas comparações internas interrompendo o laço rapidamente ao encontrar elementos já posicionados. Em contrapartida, o Shellsort, realiza testes preliminares desnecessários com saltos longos em suas primeiras fases — o que gerou um total de 700.254 comparações contra 50.247 do Insertionsort, aumentando o tempo de execução.

3 Declaração do Uso de Inteligência Artificial Generativa

Os autores declaram o uso de Inteligência Artificial Generativa (GenAI) durante o desenvolvimento deste trabalho. Especificamente, utilizou-se GenAI nas seguintes tarefas:

Tabela 2 – Resultados da execução do Insertionsort e Shellsort em um vetor com 50.000 elementos parcialmente ordenado.

Algoritmo	Comparações	Trocas	Tempo (ms)
Insertionsort	51.254	1.255	0,00007
Shellsort	701.261	1.255	0,00135

geração do gráfico de barras; implementação do arquivo Makefile; e auxílio na correção de erros de compilação.